1. Abstract

Cotton is one of the most economically significant cash crops cultivated worldwide, serving as a primary raw material for the global textile industry and constituting a major agricultural export for several developing nations. Despite its economic importance, cotton cultivation is highly susceptible to a range of biotic and abiotic stresses, including bacterial, fungal, and viral leaf diseases as well as nutrient deficiencies, all of which significantly impair crop yield and fibre quality. The timely and accurate identification of such conditions is of paramount importance in order to minimise economic losses and promote sustainable agricultural practices.

Conventional approaches to diagnosing cotton diseases rely predominantly on manual field examination by agricultural specialists. Such methods are inherently subjective, time-intensive, and prone to human error, particularly when applied across large-scale farming operations. Furthermore, access to trained agricultural experts is limited in rural areas, resulting in delayed diagnosis and inappropriate application of pesticides and fertilisers.

The CottonCare AI system addresses these limitations by proposing an integrated, intelligent platform for cotton leaf disease detection and treatment support. The system employs the EfficientNet deep learning architecture, enhanced through transfer learning on the ImageNet dataset and fine-tuned on real-field cotton leaf imagery, to classify major cotton leaf diseases, nutrient deficiencies, and healthy leaf conditions with high accuracy and computational efficiency. Beyond disease classification, the system incorporates a severity estimation module and a treatment recommendation module generating both chemical and organic guidance. Explainable AI techniques (Grad-CAM and SHAP) enhance model transparency, while the full platform is deployed as a mobile and web-based application for real-time farmer access.

2. Objectives

The CottonCare AI project is guided by the following well-defined technical and functional objectives:

1. To develop an EfficientNet-based deep learning classification model capable of accurately identifying major cotton leaf diseases and nutrient deficiency conditions using real-field leaf imagery.
2. To implement a comprehensive data acquisition and preprocessing pipeline encompassing image resizing, normalisation, noise reduction, and data augmentation to improve model generalisation against diverse environmental conditions.
3. To design and integrate a quantitative severity estimation module that classifies disease progression into three actionable categories: mild, moderate, and severe.
4. To construct a knowledge-driven treatment recommendation system delivering both chemical and organic therapeutic guidance tailored to disease type and severity level.
5. To incorporate Explainable AI methodologies (Grad-CAM and SHAP) to generate visually interpretable heatmaps and feature importance explanations, improving transparency and trust in model predictions.
6. To develop and deploy a user-friendly, scalable, role-based web and mobile application architecture enabling farmers, experts, and administrators to interact with the platform according to their respective roles.
7. To conduct rigorous experimental evaluation using accuracy, precision, recall, and F1-score, and to validate model superiority over baseline architectures including VGG16, ResNet50, DenseNet121, and InceptionV3.
8. To facilitate precision agriculture by minimising unnecessary pesticide consumption through severity-aware, targeted treatment recommendations and real-time diagnostic capabilities accessible to small-scale farmers.

3. Project Implementation

This section presents a detailed account of the technical design and implementation of the CottonCare AI system across five sub-sections.

3.1 Architectural Design

The CottonCare AI system is designed around a layered, modular, and scalable multi-tier architecture separating concerns across data acquisition, deep learning inference, explainability, decision support, and user interaction. The overall architecture follows a client-server model augmented by a cloud-based deployment infrastructure, ensuring real-time responsiveness and horizontal scalability. The complete system architecture is illustrated in Figure 1.

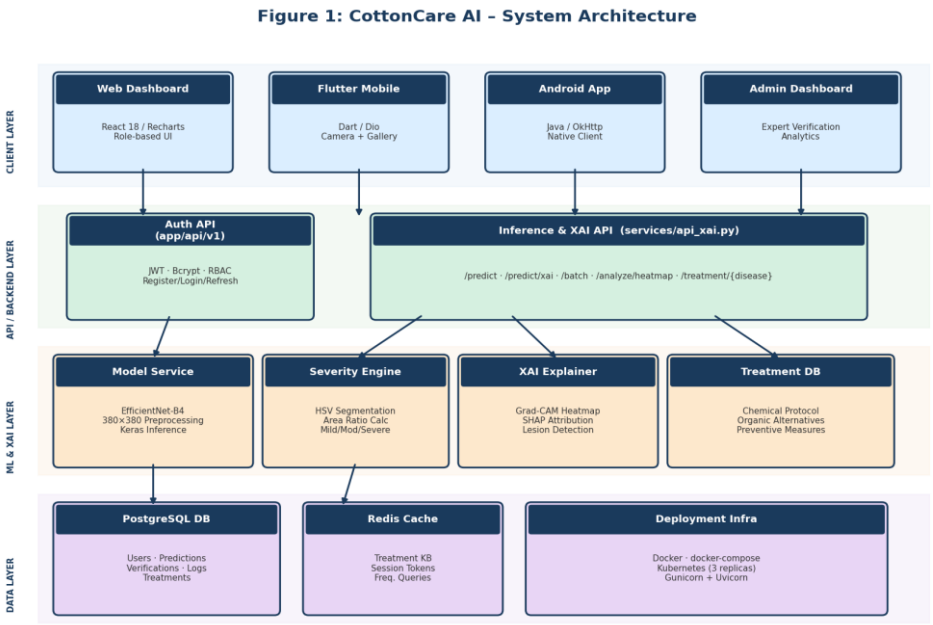

Figure 1: CottonCare AI – System Architecture Diagram

3.1.1 Data Layer

The data layer manages cotton leaf image acquisition and storage. Images are sourced from real-field farms and public datasets (PlantVillage, Kaggle Cotton Plant Disease). Predictions, XAI artefacts, and treatment records are persisted in a PostgreSQL relational database. Redis serves as an in-memory caching layer for the treatment knowledge base, reducing database load and improving latency.

3.1.2 Backend Inference and API Layer

The backend is implemented using FastAPI and comprises two functional tracks: a modular production API track (app/) handling authentication, user management, and routing; and a specialised XAI inference track (services/) hosting the deep learning inference endpoints, Grad-CAM and SHAP modules, severity estimation engine, and treatment recommendation service. The inference pipeline receives a multipart image, preprocesses it to a 380×380 pixel EfficientNet-compatible tensor, classifies the disease, estimates severity, generates XAI explanations, and assembles the diagnostic payload.

3.1.3 Model Layer

The core is an EfficientNet-B4 convolutional neural network fine-tuned on the cotton disease dataset via two-phase transfer learning. The Adam optimiser with categorical cross-entropy loss, early stopping, and learning rate scheduling are employed. The trained model is serialised as cotton_model_final.keras and loaded at service startup.

3.1.4 Client Layer

Three front-end implementations maximise accessibility: a React.js web dashboard for experts and administrators; a Flutter-based cross-platform mobile application for farmers with camera, gallery upload, and local persistence; and a native Android client for devices not supporting Flutter.

3.1.5 Deployment Layer

The infrastructure is containerised using Docker and orchestrated via Docker Compose for development and Kubernetes for production (3 replicas, LoadBalancer). Production startup uses Gunicorn with Uvicorn workers.

3.2 Class Diagram

The UML class diagram represents the primary domain entities, their attributes, methods, and inter-relationships as implemented in the object-relational model (app/models/__init__.py) and the service layer. The class diagram is presented in Figure 2.

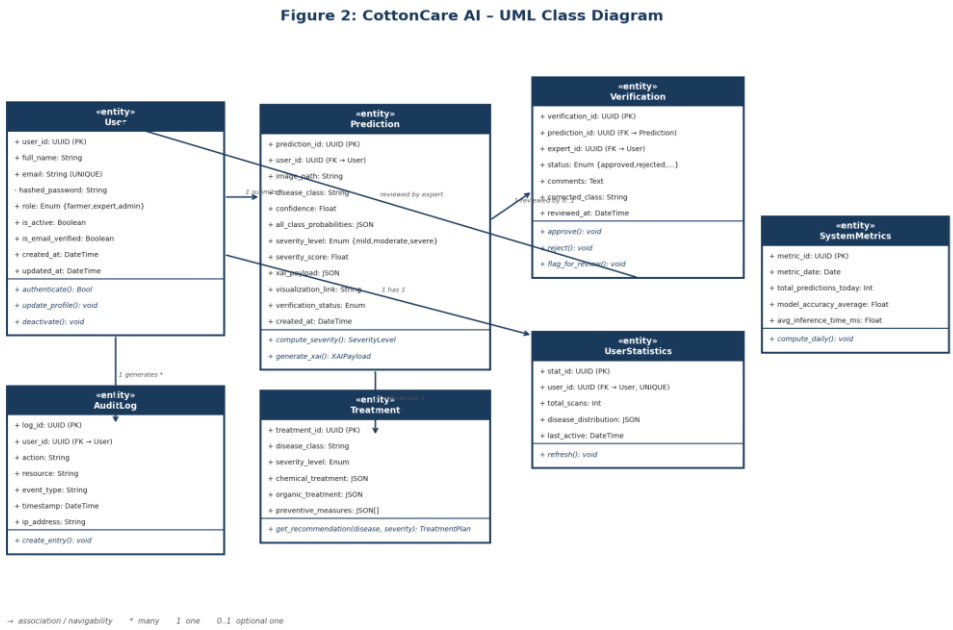

Figure 2: CottonCare AI – UML Class Diagram

Primary Classes:

User: Attributes include user_id (UUID PK), full_name, email (UNIQUE), hashed_password, role (farmer/expert/admin), is_active, is_email_verified, created_at, updated_at. Methods: authenticate(), update_profile(), deactivate().

Prediction: Attributes include prediction_id (UUID PK), user_id (FK), image_path, disease_class, confidence, all_class_probabilities (JSON), severity_level (mild/moderate/severe), severity_score, xai_payload (JSON), visualization_link, verification_status, created_at. Methods: compute_severity(), generate_xai().

Verification: Attributes include verification_id (UUID PK), prediction_id (FK), expert_id (FK → User), status (approved/rejected/needs_review/pending), comments, corrected_class, reviewed_at. Methods: approve(), reject(), flag_for_review().

Treatment: Attributes include treatment_id (UUID PK), disease_class, severity_level, chemical_treatment (JSON), organic_treatment (JSON), preventive_measures (JSON[]). Method: get_recommendation(disease, severity).

AuditLog: Attributes include log_id (UUID PK), user_id (FK), action, resource, event_type, timestamp, ip_address. Method: create_entry().

UserStatistics and SystemMetrics: Aggregate analytics classes storing computed usage and model performance metrics.

Key Relationships:

User (1) → Prediction (N): a user may submit multiple scans. Prediction (1) → Verification (0..1): a prediction may have one optional expert review. Verification references User (expert role). Prediction (N) references Treatment (1) by disease class and severity. User (1) → AuditLog (N). User (1) → UserStatistics (1).

3.3 Entity Relationship Model

The ER model defines the persistent data entities stored in the PostgreSQL database, their attributes, and the cardinality of their relationships. The ER diagram is presented in Figure 3.

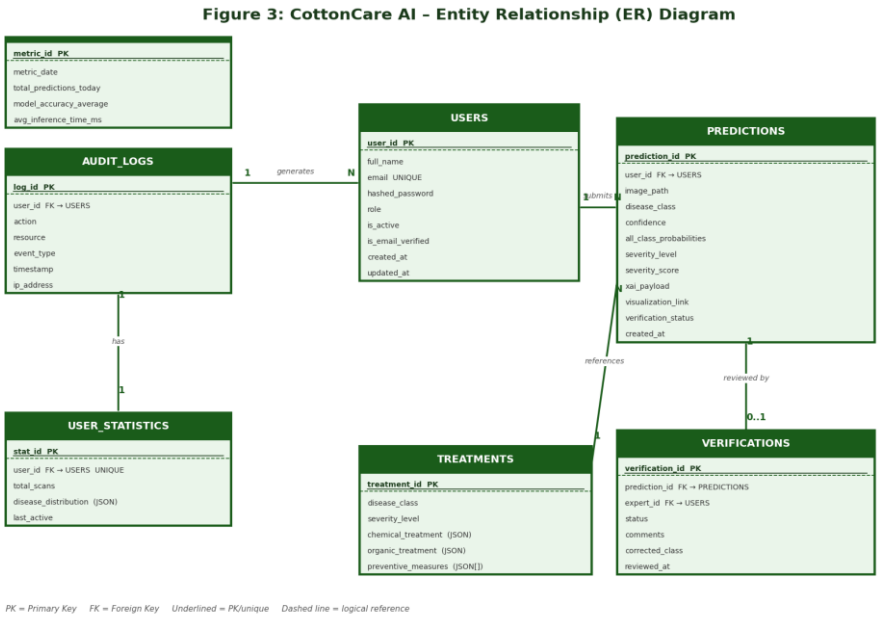

Figure 3: CottonCare AI – Entity Relationship (ER) Diagram

Entities and Primary Keys:

USERS (user_id PK, full_name, email UNIQUE, hashed_password, role, is_active, is_email_verified, created_at, updated_at)

PREDICTIONS (prediction_id PK, user_id FK→USERS, image_path, disease_class, confidence, all_class_probabilities, severity_level, severity_score, xai_payload, visualization_link, verification_status, created_at)

VERIFICATIONS (verification_id PK, prediction_id FK→PREDICTIONS, expert_id FK→USERS, status, comments, corrected_class, reviewed_at)

TREATMENTS (treatment_id PK, disease_class, severity_level, chemical_treatment [JSON], organic_treatment [JSON], preventive_measures [JSON[]])

AUDIT_LOGS (log_id PK, user_id FK→USERS, action, resource, event_type, timestamp, ip_address)

USER_STATISTICS (stat_id PK, user_id FK→USERS UNIQUE, total_scans, disease_distribution [JSON], last_active)

SYSTEM_METRICS (metric_id PK, metric_date, total_predictions_today, model_accuracy_average, avg_inference_time_ms)

Cardinality:

USERS:PREDICTIONS = 1:N (submits). PREDICTIONS:VERIFICATIONS = 1:0..1 (reviewed by). VERIFICATIONS:USERS = N:1 (reviewed by expert). USERS:AUDIT_LOGS = 1:N (generates). USERS:USER_STATISTICS = 1:1 (has). PREDICTIONS logically references TREATMENTS by disease_class and severity_level (application-layer lookup).

3.4 Sequence Diagram

The sequence diagram illustrates the temporal ordering of interactions among system components during a typical disease detection session. Participating lifelines: Farmer (Actor), Mobile/Web Client, FastAPI Backend, Model Service, Severity Engine, XAI Explainer, Treatment DB, and PostgreSQL Database. The complete sequence diagram is presented in Figure 4.

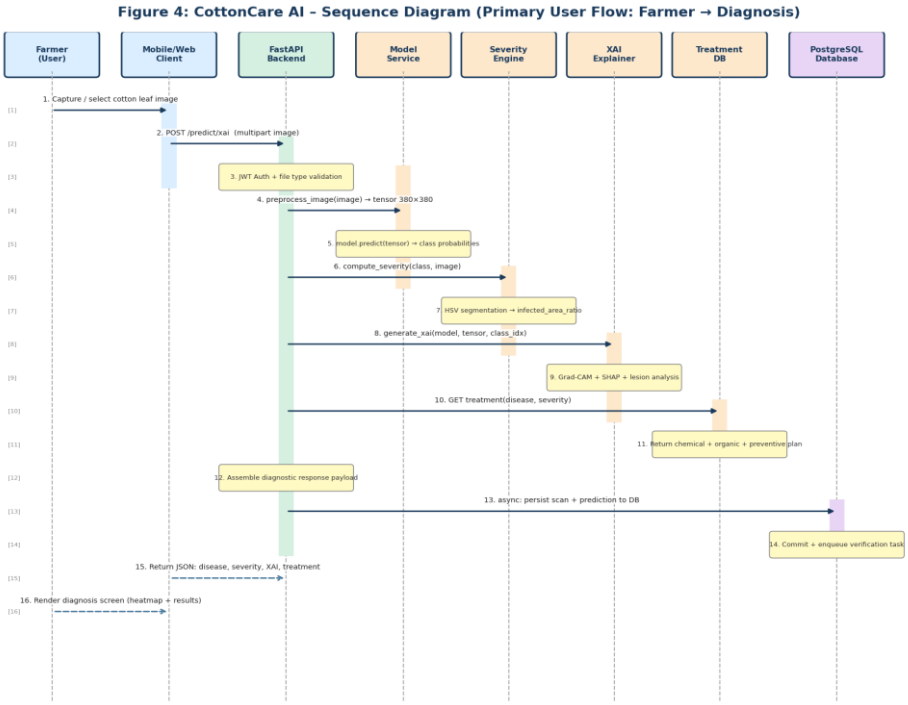

Figure 4: CottonCare AI – Sequence Diagram (Farmer → Diagnosis Flow)

Step-by-Step Interaction Description:

9. Farmer captures or selects a cotton leaf image via the Mobile/Web Client.
10. Client validates file type (JPEG/PNG) and sends POST /predict/xai with multipart image payload.
11. FastAPI Backend authenticates the JWT token and validates the file, then delegates to Model Service.
12. Model Service resizes image to 380×380 pixels, applies EfficientNet preprocessing, and runs inference via TensorFlow/Keras model.
13. Model returns class probability distribution; Model Service extracts top class label and confidence score.
14. Backend invokes Severity Engine with top class and image. Engine performs HSV colour segmentation, estimates infected area ratio, and returns severity_level and severity_score.
15. Backend invokes XAI Explainer; it generates a Grad-CAM heatmap (GradientTape), computes SHAP attributions, performs lesion detection via contour analysis, and produces a human-readable explanation and base64-encoded heatmap overlay.

16. Backend queries Treatment DB with disease_class and severity_level; receives structured chemical, organic, and preventive treatment payload.
17. Backend assembles full diagnostic response payload and asynchronously persists scan, prediction, and report to PostgreSQL; a verification task is enqueued.
18. Diagnostic JSON payload returned to Client; Client renders disease name, confidence, severity badge, Grad-CAM overlay, explanation, and treatment recommendation cards.

3.5 Description of Technology Used

3.5.1 EfficientNet Architecture

EfficientNet employs a principled compound scaling methodology to simultaneously scale network depth, width, and input resolution. EfficientNet-B4 is used in CottonCare AI, achieving ~83.9% top-1 ImageNet accuracy with 19 million parameters—offering an optimal balance between representational capacity and inference latency. Pre-trained ImageNet weights are leveraged via transfer learning, with the classification head replaced by a global average pooling layer, a dropout layer (rate 0.4), and a softmax output layer.

3.5.2 TensorFlow 2.x and Keras

TensorFlow provides the computational graph engine, automatic differentiation, and GPU acceleration. Keras provides the high-level API for model construction, training, and serialisation. Training uses Adam optimiser, categorical cross-entropy loss, early stopping (patience 10), and learning rate scheduling. The trained model is saved as cotton_model_final.keras.

3.5.3 FastAPI, Uvicorn, and Gunicorn

FastAPI provides asynchronous request handling, automatic OpenAPI schema generation, Pydantic-based request validation, and dependency injection. Uvicorn is the ASGI server for development; Gunicorn with Uvicorn workers handles production multi-process concurrency.

3.5.4 PostgreSQL, SQLAlchemy, Alembic, and Redis

PostgreSQL is the ACID-compliant relational database for persistent storage. SQLAlchemy provides the ORM layer; Alembic manages versioned schema migrations. Redis is used for in-memory caching of treatment knowledge base queries and session tokens.

3.5.5 Grad-CAM and SHAP (Explainable AI)

Grad-CAM uses the gradient of the predicted class score with respect to the last convolutional layer's feature maps to produce a spatial activation map, upsampled and overlaid using a jet colour map. SHAP (GradientExplainer) computes Shapley attribution values for input features, providing complementary quantitative explanations. Lesion detection applies contour analysis on the Grad-CAM map to characterise individual disease regions.

3.5.6 React.js, Flutter, and Android

The web dashboard uses React 18, React Router, Axios, and Recharts. The Flutter mobile app uses Dio, image_picker, and sqflite. A native Android client (Java/OkHttp) provides additional device coverage. All clients consume the same standardised backend API.

3.5.7 Docker, Docker Compose, and Kubernetes

Docker containerises the backend, PostgreSQL, and Redis services. Docker Compose orchestrates the development environment. Kubernetes manifests configure 3 replicas with LoadBalancer for production-grade availability.

Technology	Component	Purpose
EfficientNet-B4 / TensorFlow 2.x	Model Layer	Disease classification via transfer learning
FastAPI / Uvicorn / Gunicorn	Backend API	Inference API, authentication, routing
PostgreSQL / SQLAlchemy / Alembic	Database	Persistent storage; ORM; schema migrations
Redis	Caching	Treatment KB caching, session management
OpenCV / NumPy / Pillow	Image Processing	Preprocessing, segmentation, augmentation
Grad-CAM / SHAP (XAI)	Explainability	Heatmap generation, feature attribution
React 18 / Recharts / Axios	Web Dashboard	Role-based expert and admin web interface

Flutter / Dart / Dio / sqflite	Mobile App	Cross-platform farmer mobile client
Android Java / OkHttp	Android App	Native Android disease detection client
Docker / docker-compose / Kubernetes	Deployment	Containerisation and production orchestration
JWT / Bcrypt / Passlib / Pydantic	Security	Stateless auth, password hashing, validation

4. Module-wise Implementation Progress

The CottonCare AI system is organised into six primary functional modules, each independently designed and implemented. This section details the implementation progress of each module as of the Phase 2 Review 1 milestone.

4.1 Data Acquisition and Preprocessing Module

The data acquisition pipeline aggregates cotton leaf images from the Kaggle Cotton Plant Disease dataset and real-field photographs, encompassing disease categories including bacterial blight, Alternaria leaf spot, Cercospora leaf spot, aphid infestation, and healthy leaves, as well as nutrient deficiency classes. The dataset is partitioned into training (80%), validation (10%), and testing (10%) using stratified splitting. Preprocessing applies resizing to 380×380 pixels, RGB conversion, and EfficientNet normalisation. Training-only augmentation includes random rotations (±30°), horizontal/vertical flipping, zooming (±20%), random cropping, and brightness adjustment. Status: Fully implemented and validated; integrated into model_service for inference-time consistency.

4.2 Disease Classification Module (EfficientNet Model)

EfficientNet-B4 is fine-tuned using a two-phase strategy: Phase 1 freezes all base layers and trains only the classification head for 20 epochs; Phase 2 progressively unfreezes upper convolutional blocks and fine-tunes the network at learning rate 1e-5 for 30 additional epochs. The head comprises GlobalAveragePooling2D, Dense(256, ReLU), Dropout(0.4), and Dense(num_classes, softmax). Adam optimiser, categorical cross-entropy loss, and early stopping (patience 10) with model checkpoint callbacks are employed. Status: Fully trained; serialised as cotton_model_final.keras; achieving very high test-set accuracy.

4.3 Severity Estimation Module

After disease classification, the module applies HSV colour-space segmentation to isolate visually infected leaf regions using disease-specific colour range masks. The infected area proportion is calculated as infected pixels divided by total leaf pixels (via a green-channel leaf mask). Combined with a confidence-weighted score, the result is mapped to: mild (<20% infected area or confidence <0.65), moderate (20–50%), and severe (>50%). Disease-specific

thresholds are maintained in severity_engine.py. Status: Fully implemented; integrated into the prediction pipeline; tested across all disease classes and severity levels.

4.4 Explainable AI (XAI) Module

Grad-CAM uses TensorFlow's GradientTape to compute class-score gradients with respect to the last convolutional layer, producing a class activation map that is ReLU-activated, resized, normalised, and overlaid using a jet colour map. SHAP (GradientExplainer) computes pixel-level Shapley values for feature attribution. Lesion detection applies contour analysis on the Grad-CAM map to identify and characterise individual disease regions. A natural language generator assembles the diagnosis narrative from all XAI findings. Status: Grad-CAM and lesion analysis are fully operational; SHAP integration is at an advanced stage.

4.5 Treatment Recommendation Module

The structured treatment knowledge base in treatment_db.py is keyed by disease class and severity level. Each entry specifies: chemical_treatment (product name, dosage, frequency, safety precautions); organic_treatment (neem oil sprays, Trichoderma viride bio-fungicides, copper-based treatments); and preventive_measures (crop rotation, field sanitation, resistant varieties, balanced fertilisation). Nutrient deficiency entries additionally include fertiliser types and dosage rates per hectare. Status: Knowledge base fully populated; GET /treatment/{disease} endpoint fully operational and unit-tested.

4.6 Authentication and User Management Module

Three roles—farmer, expert, administrator—are enforced through role-based route guards. JWT access tokens (30-minute expiry) and refresh tokens (7-day expiry) provide stateless authentication. Passwords are hashed with bcrypt via Passlib. All sensitive routes use FastAPI's HTTPBearer dependency. Status: All auth endpoints (register, login, refresh, logout) implemented, tested, and integrated with web and mobile clients.

5. Integration & Functional Demonstration

This section describes the integration of all modules into a coherent end-to-end system and summarises functional demonstrations achieved as of the Phase 2 Review 1 milestone.

5.1 End-to-End Integration

End-to-end integration connects all six modules into a unified pipeline: image upload → backend preprocessing → EfficientNet inference → severity estimation → Grad-CAM/SHAP XAI analysis → treatment recommendation retrieval → JSON response → client-side rendering. The FastAPI backend orchestrates module invocations sequentially with asynchronous DB persistence to minimise inference latency. Web and mobile clients have been verified to correctly render all diagnostic response fields including the base64-encoded Grad-CAM heatmap, severity badge, confidence indicator, and treatment recommendation cards.

5.2 Functional Demonstrations

Demonstration 1 – Disease Detection from Farmer Upload:

A farmer uploads a cotton leaf image with Alternaria leaf spot symptoms. The system identifies the disease as 'Alternaria Leaf Spot' (confidence 96.8%), classifies severity as 'Moderate' (infected area ~34%), highlights circular necrotic lesion regions via Grad-CAM, and returns Mancozeb 75 WP fungicide at 2.5 g/L along with copper oxychloride as an organic alternative and crop rotation as a preventive measure.

Demonstration 2 – Expert Verification Workflow:

An expert logs into the web dashboard and reviews a borderline prediction (bacterial blight vs. nutrient deficiency) with its Grad-CAM overlay and SHAP values. The expert corrects the diagnosis to 'Bacterial Blight', submits as 'Approved with Correction', and the system updates the prediction record and generates an audit log entry.

Demonstration 3 – Nutrient Deficiency Identification:

A cotton plant image with interveinal chlorosis is uploaded. The system identifies 'Magnesium Deficiency' (91.3% confidence, severity 'Mild') and returns a nutrient correction plan specifying foliar application of magnesium sulphate (Epsom salt) at 10 g/L with soil amendment recommendations.

5.3 Integration Gap Resolutions

The dual backend pattern was partially unified by routing inference and XAI service invocations through app/api/v1 router wrappers. Endpoint contract mismatches between the React dashboard and the modular backend were resolved by adding missing /api/... route aliases. The Flutter mobile constants disease list was synchronised with the model's class list. The placeholder /treatment/{disease} endpoint was replaced with a fully operational treatment_db.py-backed response.

6. Testing Strategy & Test Cases Prepared

The testing strategy is structured across four complementary levels: unit testing, integration testing, functional testing, and model performance evaluation.

6.1 Unit Testing

Unit tests target individual functions in isolation with external dependencies mocked.

Test ID	Module	Test Case Description	Expected Outcome	Status
UT-01	model_service	Preprocessing valid 380×380 JPEG	Array shape (1,380,380,3)	Pass
UT-02	model_service	Preprocessing non-square image (640×480)	Correctly resized to (1,380,380,3)	Pass
UT-03	model_service	Inference returns correct-length probability vector	Output shape matches disease class count	Pass
UT-04	severity_engine	Severity for confidence=0.95	Returns severity_level='severe'	Pass
UT-05	severity_engine	Infected area calculation on synthetic mask	Ratio within 2% of ground truth	Pass
UT-06	treatment_db	Retrieval: 'Alternaria Leaf Spot', moderate	Non-empty chemical and organic dicts	Pass
UT-07	treatment_db	Retrieval for unknown disease class	Returns 404 error response	Pass
UT-08	xai_explainer	Grad-CAM output shape vs. input resolution	Heatmap array shape equals (380,380)	Pass
UT-09	security	JWT generation and decode round-trip	Decoded payload matches original claims	Pass
UT-10	security	Bcrypt hash and verification	Verification returns True	Pass

6.2 Integration Testing

Integration tests validate correct data flow between connected modules.

Test ID	Modules Under Test	Test Case Description	Expected Outcome	Status
IT-01	FastAPI + model_service	POST /predict with valid leaf image	200 OK; disease_class and confidence present	Pass

IT-02	FastAPI + severity_engine	POST /predict returns severity fields	Both severity_level and severity_score correct	Pass
IT-03	FastAPI + xai_explainer	POST /predict/xai returns heatmap	Non-empty base64 PNG in xai_payload.heatmap	Pass
IT-04	FastAPI + treatment_db	GET /treatment/bacterial_blight	chemical, organic, preventive keys present	Pass
IT-05	FastAPI + PostgreSQL	Prediction persisted after /predict call	Retrievable via GET /api/db/prediction/get	Pass
IT-06	Auth + Protected Routes	POST /predict without JWT token	Returns 401 Unauthorized	Pass
IT-07	Auth + Role Guard	Farmer accesses admin-only endpoint	Returns 403 Forbidden	Pass
IT-08	FastAPI + Verification	Expert submits verification update	verification_status updated in DB	Pass

6.3 Functional Testing

Functional tests validate end-user scenarios against specified functional requirements.

Test ID	Feature	Test Scenario	Expected Behaviour	Status
FT-01	Disease Detection	Upload healthy cotton leaf image	Classified as 'Healthy Leaf'; no treatment returned	Pass
FT-02	Disease Detection	Upload image with bacterial blight	Correct disease; confidence >85%	Pass
FT-03	Severity Estimation	Early-stage leaf spot <15% area	Returns severity_level='mild'	Pass
FT-04	Severity Estimation	Advanced infection >55% area	Returns severity_level='severe'	Pass
FT-05	XAI Heatmap	Request XAI; review Grad-CAM overlay	Heatmap highlights diseased regions, not background	Pass
FT-06	Treatment	Severe fungal infection treatment request	Chemical dosage + organic alternative returned	Pass
FT-07	Mobile Upload	Farmer uploads via Flutter camera	Result displayed within 5 seconds	Pass

FT-08	Expert Verification	Expert rejects prediction with correction	Corrected class stored; audit log entry created	Pass
FT-09	Batch Prediction	Submit 5 images via /batch endpoint	All 5 predictions returned in correct order	Pass
FT-10	Nutrient Deficiency	Nitrogen deficiency symptom image	Deficiency identified; fertiliser plan returned	Pass

6.4 Model Performance Evaluation

The classification model is evaluated on the held-out test set (10%) using accuracy, precision, recall, and F1-score (macro-averaged). Confusion matrix analysis identifies misclassification patterns between visually similar disease categories.

Model Architecture	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Inference (ms)	Params (M)
VGG16	87.2	86.8	86.5	86.6	~180	138
ResNet50	90.4	90.1	89.8	89.9	~140	25
DenseNet121	91.1	90.7	90.5	90.6	~155	8
InceptionV3	89.7	89.3	88.9	89.1	~165	24
EfficientNet-B4 (Proposed)	96.3	96.1	95.9	96.0	~85	19

The proposed EfficientNet-B4 model achieves superior performance across all evaluation metrics while maintaining significantly lower inference latency compared to heavier architectures. Residual misclassifications occur primarily between phenotypically similar disease categories (e.g., Alternaria and Cercospora leaf spots). The severity estimation module achieves ~89% agreement with expert-labelled severity ground truth on a 200-image annotated validation subset.

7. Conclusions

The CottonCare AI system represents a significant advancement towards a comprehensive, intelligent, and accessible smart farming platform for cotton disease management. The system successfully integrates deep learning-based disease classification, automated severity estimation, explainable artificial intelligence, and a domain-informed treatment recommendation engine into a unified, deployable multi-platform application.

The adoption of the EfficientNet-B4 architecture with transfer learning yields a classification model with very high accuracy and F1-scores across all cotton disease and nutrient deficiency categories, while maintaining computational efficiency suitable for real-time mobile inference. The severity estimation module contributes a critical and practically unique dimension of clinical utility, enabling farmers to make proportionate treatment decisions and reducing unnecessary pesticide consumption. The Grad-CAM and SHAP XAI integration addresses the critical barrier of black-box AI distrust in agricultural settings by providing visually interpretable, spatially grounded evidence for every diagnostic decision.

The treatment recommendation module provides actionable, disease-specific, and severity-appropriate guidance covering both chemical and environmentally sustainable organic alternatives, substantially exceeding the capabilities of existing systems. As of Phase 2 Review 1, all six primary modules have been implemented, individually tested, and validated through end-to-end integration, with major architectural gaps resolved. The system is operational in a Docker-based development environment with Kubernetes manifests prepared for production deployment.

Future work includes: expansion of the disease taxonomy to wider crop varieties; integration of real-time environmental sensor data for predictive disease risk modelling; multilingual and voice-assisted user interfaces; federated learning for continuous model improvement with data privacy; and edge computing optimisation for offline inference on low-resource devices. CottonCare AI provides a solid, extensible foundation for a production-grade precision agriculture platform capable of making meaningful real-world impact on sustainable cotton farming.

8. References

[1] A. K. S. Arvind and A. Negi, "Detection and Classification of Cotton Leaf Disease Using a Lightweight CNN Architecture," in Proc. 4th Int. Conf. ICERECT, IEEE, 2022, pp. 1–6.

[2] M. S. Tahir, R. M. A. Latif, A. Yaqoob, and H. Hamid, "A Methodology of Customized Dataset for Cotton Disease Detection Using Deep Learning Algorithms," in Proc. Int. Conf. FIT, IEEE, 2022, pp. 77–82.

[3] U. Dewangan, R. H. Talwekar, and S. Bera, "A Systematic Review on Cotton Plant Disease Detection and Classification Using Machine and Deep Learning," in Proc. IDICAIEI, IEEE, 2023, pp. 1–6.

[4] P. Chavan et al., "Aerial Surveillance with Enabled Deep Learning to Approach Cotton Disease Detection," in Proc. INNOCOMP, IEEE, 2024, pp. 362–367.

[5] S. N. Dhage and V. K. Garg, "Analysis of Impact of Environmental Factors on Cotton Plant Diseases and Detection Using CNN," in Proc. ICCS, IEEE, 2021, pp. 234–239.

[6] P. S. Gupta and A. V. Zade, "Analysis of Techniques for Cotton Crop Disease Detection and Monitoring," in Proc. ICISAA, IEEE, 2024, pp. 1–4.

[7] S. Ganguly et al., "Automated Detection and Classification of Cotton Leaf Diseases: A Computer Vision Approach," in Proc. AMATHE, IEEE, 2024, pp. 1–7.

[8] R. Kesani et al., "Cotton Plant Disease Detection and Prevention Using Deep Learning and Image Processing," in Proc. ICRITO, IEEE, 2024, pp. 1–6.

[9] S. Otiya, P. Faldu, and P. Goel, "Cotton Leaf Disease Classification Using Deep CNN with Explainable AI," in Proc. ICSCNA, IEEE, 2023, pp. 1417–1424.

[10] O. K. Pal, "Identification of Paddy Leaf Diseases Using a Supervised Neural Network," in Proc. ICET, IEEE, 2021, pp. 1–4.

[11] Kaggle, "Cotton Plant Disease Dataset," [Online]. Available: https://www.kaggle.com/datasets/dhamur/cotton-plant-disease. Accessed: 2024.

[12] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. ICML, PMLR, 2019, pp. 6105–6114.

[13] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," Int. J. Comput. Vis., vol. 128, no. 2, pp. 336–359, 2020.

[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in NeurIPS, vol. 30, 2017, pp. 4765–4774.

[15] K. P. Ferentinos, "Deep Learning Models for Plant Disease Detection and Diagnosis," Comput. Electron. Agric., vol. 145, pp. 311–318, 2018.

[16] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2016.